SASTI DAWAI

Machine Learning & AI Product Requirements Document

Scan a medicine strip. Find the exact same salt for less.

Version 1.0 • 13 October 2026

Derived from Sasti-Dawai-PRD.pdf v1.0

CORE RULE: AI READS. VERIFIED DATA CHECKS. DETERMINISTIC CODE DECIDES.

Document control

Project: Sasti Dawai — Scan a medicine strip. Find the exact same salt for less.

Document: Machine Learning & AI Product Requirements Document (ML PRD)

Version: 1.0 | **Date:** 13 October 2026 | **Parent document:** Sasti-Dawai-PRD.pdf, especially sections 6, 8, 9 and 10.

This document defines only the ML/AI and data-resolution layer. It is written for both an AI coding agent and the student team. It explains what to build, why each component exists, how it should behave, and how to test it.

Core principle: use AI to read messy medicine packaging; use verified data and deterministic code to decide whether two products are equivalent and to calculate prices. Never ask an LLM to invent a medicine match, price, dose, or safety conclusion.

1. Purpose and boundaries

1.1 The ML system's job

Given one to three photos of an Indian medicine strip or box, the system should extract visible product information, resolve the brand to a verified catalogue record and canonical ingredient identities, cross-check what was read, and return evidence-backed structured data to the application.

1.2 In scope

- Image quality checks: blur, darkness, glare/clipping, and basic framing feedback.
- Barcode/QR decoding when a code is present and supported.
- Medicine-pack text extraction using a pretrained vision-language model (VLM), with OCR fallback/cross-check.
- Strict JSON validation and per-field confidence tracking.
- Brand/product resolution using exact lookup, lexical fuzzy matching, and optionally embedding/vector retrieval.
- Ingredient/salt and strength normalisation, including fixed-dose combinations.
- Evidence comparison and a confidence gate that can stop unsafe downstream matching.
- Evaluation scripts, error reports, latency/cost tracking, and privacy-aware feedback.

1.3 Out of scope

- Training a large image model from scratch or fine-tuning a large model during the initial build.
- Diagnosing a condition, recommending a dose, advising a patient to switch, or interpreting handwritten prescriptions.
- Letting an LLM decide medical equivalence or generate prices/stock/availability.
- Treating semantic similarity alone as proof that two medicines are equivalent.

The existing parent PRD's product scope is mobile-first, English/Hindi, with an exact-match-only safety model and an explicit doctor/pharmacist confirmation step. The ML subsystem must preserve those constraints.

2. System overview

2.1 End-to-end flow

- 1. Capture: user takes a photo; the client checks basic image quality.
- 2. Decode: try barcode/QR decoding where available.
- 3. Extract: send acceptable images to a vision model and request schema-constrained JSON.
- 4. OCR cross-check: read visible text with an OCR engine when available; compare important fields.
- 5. Validate: reject malformed output, unsupported values, impossible units, or missing critical fields.
- 6. Resolve: search the curated medicine catalogue using exact name, fuzzy text, and optional embeddings.
- 7. Verify: compare the model's printed composition against the catalogue record; compute a confidence/risk decision.
- 8. Human confirmation: show uncertain fields to the user for correction/confirmation.
- 9. Deterministic matching: a separate rules engine assigns EXACT, CLOSE, or NONE. This is not an ML decision.

• 10. Explain: optionally use an LLM to simplify verified facts in English/Hindi; inject numbers and the mandatory disclaimer in code.

2.2 Component responsibility table

Component	Recommended approach	Owns / does not own
Image quality	OpenCV + simple image statistics	Can request retake; cannot identify a medicine.
Barcode/QR	Barcode decoder library	Provides decoded payload only; catalogue lookup must verify it.
Text extraction	Pretrained vision-language model with JSON schema	Transcribes visible evidence; must not infer unreadable salts.
OCR fallback	PaddleOCR or another tested OCR engine	Independent text evidence; not a medicine-equivalence engine.
Resolver	PostgreSQL exact lookup + pg_trgm/RapidFuzz + optional pgvector	Finds likely catalogue records; does not certify equivalence.
Safety gate	Deterministic Python rules	Blocks uncertain or contradictory records.
Exact matcher	Deterministic rules from parent PRD	Only component allowed to issue EXACT tier.
Explainer	Template first; optional constrained LLM	Rephrases verified facts only; no medical advice.

A Vision-Language Model (VLM) is an AI model that can process both images and text instructions. OCR means Optical Character Recognition: software that turns text in an image into machine-readable characters.

3. ML design decisions

3.1 Use pretrained models; do not train from scratch

The first version should use a hosted vision-capable model with structured-output/JSON-schema support, plus a tested OCR fallback such as PaddleOCR. This is appropriate because the difficult part is reading small, curved, reflective, multilingual packaging, while the project does not have a large labelled image dataset. A pretrained model has already learned general visual and language patterns.

Do not train on the medicine catalogue CSV as though each row were an image-training example. A catalogue is structured product data, not labelled photographs. Use it for lookup, normalization, candidate retrieval, and validation.

3.2 Why a deterministic matcher is mandatory

An LLM may sound confident while being wrong. Medicine equivalence has strict rules: same complete active-ingredient set, strength per unit, dosage form, release type, and route. The model may read text, but ordinary application code must compare canonical fields and determine the tier.

3.3 Confidence is a safety signal, not a guarantee

A confidence score helps decide when to ask the user for confirmation. It does not prove correctness. The system must combine field confidence with independent evidence and evaluate thresholds on labelled examples. If evidence conflicts or a critical field is uncertain, fail closed: do not return an EXACT match.

4. Input and image quality

4.1 Input contract

- Accept one to three images of a medicine strip or box, ideally showing brand and composition.
- Compress each uploaded image to at most 1.5 MB, as specified by the parent PRD.
- Do not require a user account. Do not retain source images by default.
- Reject or safely handle unsupported file types, empty images, oversized uploads, and non-medicine images.

4.2 Quality checks

Check	Meaning	Initial implementation
Blur	Text edges are smeared or out of focus.	Variance of the Laplacian as a lightweight blur indicator; tune on real photos.
Too dark/bright	The image lacks readable detail.	Brightness histogram or mean/percentile intensity.
Glare/clipping	Foil reflection or saturated regions hide text.	Estimate unusually bright clipped regions; treat as a heuristic.
Framing/size	Pack is too small or partly outside frame.	Simple guidance first; optional crop/region detector later.

These are heuristics, not a trained classifier. Their thresholds depend on camera and lighting, so tune them using real strip photographs. Do not reject an image solely because one metric is imperfect; combine checks and let the extractor/evidence checks decide whether fields are readable.

4.3 User-facing behavior

- If quality is poor, return a short actionable message such as “Hold steady”, “Move closer”, or “Avoid reflection”.
- Allow retake, another side/photo, gallery upload, or manual search.
- Never silently transform a bad image into a confident medicine identity.

5. Extraction model and output schema

5.1 Extraction rules

- Transcribe only information visible on the pack or decoded from a verifiable code.
- Never infer a salt from a brand name. Brand-to-salt mapping happens later against the catalogue.
- If a field is unreadable or absent, return null or the defined unknown value.
- Return strict JSON only; validate it with Pydantic before using it.
- Give a confidence score from 0.0 to 1.0 for every critical field. Treat model-reported confidence as an estimate, not calibrated probability.
- If the image is not a medicine pack, set is_medicine_pack=false.

5.2 Required fields

Field	Type / example	Why it matters
is_medicine_pack	boolean	Rejects irrelevant photos.
brand_name	string or null	Initial catalogue lookup.
manufacturer	string or null	Supporting disambiguation evidence.
printed_composition	array of ingredient + strength + unit + denominator	Cross-check against catalogue; preserve combination ingredients.
dosage_form	controlled value or unknown	Tablet, capsule, syrup, suspension, injection, gel, cream, drops, inhaler, other.
release_type	IR/SR/ER/DR/MR/ODT/chewable/dispersible/unknown	Release characteristics can make products non-equivalent.
route	controlled value or unknown	Oral, topical, ophthalmic, parenteral, etc., when available.
pack_size	count + unit or null	Normalises price per tablet/ml/g/vial.
mrp_inr	number or null	Printed pack price; never invent.
expiry	YYYY-MM or null	Can support an expiry warning if readable.
rx_schedule	H/H1/X/OTC/unknown	Supporting label information; not permission to dispense.
legibility	good/partial/poor	Overall reading quality.
field_confidence	0–1 per critical field	Human-confirmation and evaluation signal.

5.3 Example output


```
{
  "is_medicine_pack": true,
  "brand_name": "Example Brand SR 500",
  "manufacturer": null,
  "printed_composition": [
    {
      "salt_text": "Metformin Hydrochloride",
      "strength_value": 500,
      "strength_unit": "mg",
      "per": "tablet"
    }
  ],
  "dosage_form": "tablet",
  "release_type": "SR",
  "route": "oral",
  "pack_size": {"count": 10, "unit": "tablets"},
  "mrp_inr": null,
  "expiry": null,
  "rx_schedule": "unknown",
  "legibility": "partial",
  "field_confidence": {
    "brand_name": 0.94,
    "composition": 0.88,
    "strength": 0.73,
    "dosage_form": 0.98,
    "release_type": 0.81,
    "mrp_inr": 0.0
  }
}
```


This is an illustrative example, not a real product record. Notice that the unreadable MRP is null rather than guessed.

5.4 Prompt template for the extraction model

```
SYSTEM:
You read photographs of Indian medicine strips and boxes.
Transcribe ONLY what is printed and visibly supported by the image.

Rules:
1. Never infer or guess a salt from the brand name.
2. If a value is unreadable or absent, return null (or the schema's unknown value).
3. Preserve every ingredient in combination medicines; do not omit ingredients.
4. Preserve the printed strength and denominator (for example, mg/tablet or mg/5 ml).
5. Do not recommend substitutes, doses, diagnoses, or treatment.
6. Return JSON only and conform exactly to the supplied schema.
7. Include a 0–1 confidence estimate for every critical field.
8. If the image is not a medicine pack, set is_medicine_pack=false.

USER:
Extract the visible fields from the supplied medicine-pack image(s).
Use the schema supplied by the application. Do not fill gaps with medical knowledge.
```

Use temperature 0 where supported and structured output constrained by the schema. These settings improve consistency but do not eliminate hallucinations; validation and cross-checks remain required.

6. OCR, barcode and evidence fusion

6.1 OCR fallback

OCR provides a second reading of visible characters. It is useful when the vision model is uncertain or when a pack has small printed text. Compare OCR output with the VLM output for critical fields such as brand, strength, composition, release marker, MRP, and expiry.

OCR disagreement is not automatically resolved by choosing whichever output looks more plausible. Preserve both pieces of evidence and mark the field as disputed. The user must confirm it or provide a clearer image.

6.2 Barcode/QR

Attempt barcode/QR decoding when present. A decoded identifier is evidence for product lookup, not a guarantee that the code is genuine, current, or correctly associated with the photographed pack. Resolve it to a trusted catalogue record and compare that record with the visible packaging.

6.3 Evidence table

Evidence source	Example	How to use it
VLM	Reads “500 mg SR”	Primary visual extraction; field confidence recorded.
OCR	Reads “500 mg” but not SR	Independent text signal; missing release marker remains uncertain.
Barcode/QR	Returns a product identifier	Look up identifier in trusted source and compare visible pack.
Catalogue	Verified brand record says 500 mg IR	Contradiction with “SR” blocks automatic matching.
User correction	User confirms visible strength	Record as user-confirmed input; retain provenance.

Evidence fusion means combining signals from multiple sources. In v1, implement it with transparent rules rather than a complicated learned ensemble. For example, critical-field agreement raises confidence; a composition or release-type contradiction forces needs_confirmation=true.

7. Brand and medicine resolver

7.1 Goal

Map noisy text from the photograph to one or more verified catalogue records. Brand names may have spelling variants, OCR errors, abbreviations, suffixes such as SR/ER, and similar-looking names.

7.2 Search sequence

- Exact normalized lookup: lowercase, trim spaces, normalize punctuation and common typography without deleting meaningful tokens.
- Lexical fuzzy search: use PostgreSQL pg_trgm or RapidFuzz to find spelling-near candidates.
- Optional vector search: embed product names, salt aliases, and carefully selected descriptive text; use pgvector to retrieve semantically similar candidates.
- Re-rank candidates using explicit evidence such as brand-token similarity, manufacturer, strength, form, release type, and composition agreement.
- Return a ranked candidate list with match evidence. Never let the highest score automatically become a verified match if critical fields conflict.

7.3 What are fuzzy and vector search?

Fuzzy search compares character patterns, so it can find a likely spelling match when OCR reads “Metformln” instead of “Metformin”. Vector search converts text into numeric embeddings and retrieves text with similar learned meaning. Fuzzy search is usually better for spelling noise; vector search can help with aliases and varied wording. Neither proves medical equivalence.

7.4 Resolver response

```
{
  "status": "candidate_found",
  "candidates": [
    {
      "product_id": "catalogue-id",
      "canonical_brand": "verified catalogue name",
      "lexical_score": 0.93,
      "vector_score": null,
      "evidence": ["brand_tokens_match", "strength_matches"],
      "critical_conflicts": [],
      "requires_confirmation": true
    }
  ]
}
```

Scores above are illustrative. The implementation must define score ranges and calibrate thresholds using the team's labelled brand-resolution set. A missing vector model is not a blocker: exact + fuzzy lookup is the required baseline.

7.5 Canonical ingredient IDs

Normalize ingredient names to stable database IDs and aliases. Store combination medicines as a set of all active ingredients, each with its own strength and denominator. Do not compare only the first ingredient or a concatenated display string.

8. Normalisation and deterministic safety boundary

8.1 Normalisation

- Units: normalize equivalent units consistently (for example, 1 g = 1000 mg; 1 mg = 1000 mcg) while preserving the original printed text.
- Denominators: distinguish mg per tablet from mg per 5 ml. Never compare different denominators without a validated conversion.
- Form: use controlled values such as tablet, capsule, syrup, suspension, injection, gel, cream, drops, inhaler, other.
- Release type: distinguish IR, SR, ER, XR, DR/EC, MR, ODT, chewable, dispersible, and unknown.
- Combination medicines: sort canonical ingredient-strength pairs so order does not matter, but every pair must match.
- Salt forms: do not assume two salt forms are interchangeable. Require an explicitly clinician-approved equivalence rule where the PRD permits one.

8.2 Exact match contract

The ML subsystem must provide clean, verified fields to the rule engine. The rule engine alone may label a candidate EXACT, CLOSE, or NONE.

Tier	Rule boundary	Savings?
EXACT	Same complete active ingredient set, same strength per unit, same dosage form, same release type, and same route; all critical evidence verified.	Yes, using trusted dated prices.
CLOSE	Related composition but a material difference exists, such as different strength/release type, or an approved-but-not-exact case requiring professional confirmation.	No.
NONE	No verified candidate, unresolved brand, unreadable critical field, or any unsafe/unsupported comparison.	No.

Safety invariant: uncertain is not equivalent. A failed or low-confidence extraction must not be converted into an EXACT match by fuzzy search, vector similarity, or an LLM explanation.

8.3 Special safety cases

- SR/ER/XR/MR versus immediate release: never EXACT.
- Fixed-dose combinations: all ingredients and strengths must match.
- Different strength: CLOSE at best; never suggest splitting or doubling tablets.
- Narrow therapeutic index medicines: display the prominent doctor warning and suppress celebratory savings emphasis as defined in the parent PRD.
- Brand and printed composition disagree: block automatic matching.
- Expired pack: show the expiry warning if the date is confidently read.
- No match: say no verified exact match was found; do not fabricate one.

9. Confidence and abstention policy

9.1 Confidence components

- Field confidence: model estimate for brand, composition, strength, form, release type, MRP, and expiry.
- Cross-check agreement: whether VLM, OCR, barcode record, and catalogue agree on critical fields.
- Resolver score: similarity between extracted text and a catalogue candidate.
- Critical conflict flags: missing strength, incomplete combination, unknown release type, or contradiction.

9.2 Gate behavior

Implement an explicit state machine rather than a single opaque confidence number:

State	Condition	Allowed action
REJECT_IMAGE	Not a medicine pack or invalid file	Stop extraction/matching; ask for a medicine-pack photo.
RETAKE_REQUIRED	Quality too poor to read critical fields	Give capture guidance.

State	Condition	Allowed action
NEEDS_USER_CONFIRMATION	Critical fields have low confidence or evidence conflicts	Show highlighted fields; user must confirm/edit.
RESOLVED	Candidate is supported by evidence and critical fields are confirmed	Pass canonical fields to deterministic matcher.
UNRESOLVED	No trustworthy catalogue candidate	Manual search or pharmacist confirmation; return NONE.

Do not hard-code a universal threshold such as 0.80 and assume it is safe. Select thresholds using the golden set, inspect precision/recall and false positives, and favor abstention when the evidence is weak. Document the chosen threshold and the evidence behind it.

9.3 Why abstention matters

Abstention means the system deliberately declines to make a claim when it lacks enough evidence. For medicine matching, a false exact match can cause harm; an honest “could not verify” result is preferable to a confident but unsafe guess.

10. Data plan and dataset handling

10.1 Data roles

Data	Purpose	Important limitation
Official Jan Aushadhi/PMBI product list	Generic catalogue and official listed prices	Store source, retrieval date, raw file checksum, and price date.
Curated brand-to-salt mapping	Resolve the scanned brand to canonical ingredients	Verify high-impact mappings; do not treat scraped data as unquestioned truth.
Indian Medicine Dataset (reported in team context as ~253,973 records)	Potential source for brand/composition/form/pack/price candidate records	Check licence, provenance, duplicate rows, stale prices, and correctness before use.
Jan Aushadhi CSV (reported in team context as ~2,439 rows)	Structured generic product/price reference	Validate columns and refresh against official source.
Golden photo set (50–100 packs)	Measure image extraction and resolver performance	Use no patient data; hand-label ground truth.
Correction feedback	Find recurring errors and improve evaluation examples	Do not automatically train or change thresholds from unreviewed feedback.

The record counts above are the previously discussed dataset sizes, not a claim that the files have been freshly inspected in this task. Preserve original files and source metadata; never overwrite raw data during cleaning.

10.2 Data preparation

- Store each raw dataset unchanged and record its source URL/name, retrieval date, row count, licence notes, and SHA-256 checksum.
- Create a reproducible ETL script (Extract, Transform, Load): parse raw rows, normalize column names, clean whitespace, standardize units/forms/release labels, and retain the original raw values.
- Deduplicate carefully using identifiers and normalized product attributes. Similar names are not necessarily duplicates.
- Represent ingredients in a separate relation/table so combination products can have multiple ingredients and strengths.
- Keep product price, source, and price date together. Never treat a stale price as a current price.
- Log rows that could not be parsed rather than silently dropping them.

10.3 Dataset leakage and training

No supervised image-model training is required for v1. Do not randomly split near-duplicate photos of the same pack across train and test if a later model is trained; that would inflate performance. For future model training, split by product/brand or capture session where appropriate, keep the test set untouched, and report the split strategy.

11. Evaluation and release gates

11.1 Golden set

Create 50–100 labelled photos of medicine strips/boxes. Include foil-only strips, boxes, curved packaging, glare, low light, small text, Hindi/English, combination medicines, and examples where critical text is intentionally hard to read. Label visible text manually. Ask a clinician contact to review the 20 highest-risk or most important examples when possible. Do not use real patient data.

11.2 Required metrics

Metric	Definition	Initial target from parent PRD
Per-field extraction accuracy	Correct field values divided by labelled examples for that field.	At least 90% for brand, salt, strength, and form on 50+ photos.
Brand resolver top-1 accuracy	Correct catalogue record ranked first.	At least 95% on curated brand list.
Brand resolver top-3 accuracy	Correct record appears among first three candidates.	Report and track; establish a target after baseline.
Exact-match false positives	Cases incorrectly labelled EXACT.	Zero on the golden set; any observed case blocks release.
Exact-match precision	Correct EXACT predictions divided by all EXACT predictions.	Target 100% on the golden set; small samples do not prove universal safety.
Recall	Relevant correct cases found divided by all relevant cases.	Report alongside precision; never improve recall by allowing unsafe matches.
Latency p50/p95	Median and 95th-percentile scan-to-result time.	At most 6 s / 10 s on 4G, per parent PRD.
Cost per scan	Average model/API cost per scan.	Measure and document; no numeric budget supplied in parent PRD.
Failure/abstention rate	Share of scans needing retake, confirmation, or no match.	Track by image condition and field; do not optimize by hiding uncertainty.

11.3 Error taxonomy

- Wrong brand or wrong catalogue candidate.
- Misread ingredient, missing ingredient in a combination, or wrong strength/unit.
- Wrong release type (for example SR read as IR).
- OCR/VLM disagreement not caught by the gate.
- False exact match from fuzzy/vector similarity.
- Image quality gate misses glare/blur or rejects a usable image.
- Wrong MRP, pack count, or expiry.

11.4 Evaluation workflow

- Freeze a versioned labelled set and keep its ground truth separate from model output.
- Run the evaluation script on every meaningful prompt/model/resolver/threshold change.
- Report per-field accuracy, resolver top-1/top-3, exact-match precision and false positives, latency, API cost, and failure cases.
- Inspect errors by category; fix the two most frequent/high-risk error types before the demo.
- Manually review every false EXACT result. Release is blocked until its cause is understood and fixed.
- Publish a small evaluation summary and limitations on the app's data/about page.

12. Implementation architecture and API contract

12.1 Recommended stack

- Python 3.11+ (or the version supported by deployment), FastAPI, Pydantic v2, pytest.
- OpenCV for image preprocessing/quality heuristics.
- A vision-capable hosted model with structured JSON output for primary extraction.
- PaddleOCR or another benchmarked OCR engine as fallback/cross-check.
- PostgreSQL with pg_trgm for lexical similarity and pgvector as an optional vector-retrieval layer.
- RapidFuzz for local fuzzy matching if useful; do not implement duplicate fuzzy systems without a measured reason.
- Structured JSON logs and a generated Markdown/HTML evaluation report.

12.2 Suggested modules

```
ml/
  app/
    api/routes_scan.py
    schemas/extraction.py
    pipeline/orchestrator.py
    vision/quality.py
    vision/extractor.py
    vision/ocr.py
    vision/barcode.py
    resolver/normalize.py
    resolver/candidates.py
    resolver/confidence.py
    safety/gates.py
    evaluation/run_eval.py
    evaluation/metrics.py
    data/etl.py
  prompts/extract_medicine.txt
  tests/
    test_schema.py
    test_normalization.py
    test_resolver.py
    test_safety_gates.py
    test_exact_match_boundary.py
  reports/
  README.md
  MODEL_CARD.md
```

12.3 API outline

```
POST /v1/scan
Input: multipart images (1–3), optional language, optional request ID.
Output:
{
  "status": "needs_confirmation",
  "extraction": { "...": "validated fields" },
  "field_status": {
    "brand_name": "high",
    "composition": "medium",
    "release_type": "low"
  },
  "candidate_products": [],
  "warnings": ["Release type could not be verified"],
  "next_action": "confirm_or_retake",
  "model_version": "configured-version",
  "latency_ms": 1234
}
```

The API must never return an EXACT match merely because extraction succeeded. Matching happens only after validation, resolution, and the safety gate. Prices and exact-match tiers must be sourced from the database and deterministic rules, not from generated text.

12.4 Reliability behavior

- Model timeout/API failure: return a recoverable error and offer retake/manual search; do not fabricate a result.
- OCR unavailable: continue only if remaining evidence is sufficient; otherwise ask for confirmation.
- Database unavailable: no verified match or current price can be claimed.
- Malformed model JSON: validate, retry once if useful, then fail safely.
- Log model/provider/version, timings, status, and non-sensitive error codes; do not log photos or unnecessary personal data.

13. Explainer AI and guardrails

13.1 Explainer role

The explainer turns verified structured facts into two or three simple sentences in English or Hindi. It is a user-interface writing layer, not a medical decision-maker.

13.2 Allowed input

- Confirmed brand and canonical ingredient names.
- Verified strength/form/release type.
- Database-provided product prices and price dates.
- The deterministic tier and its reason.
- Kendra information returned by the application, if available.

13.3 Prohibited output

- No diagnosis, dosage, treatment advice, or recommendation to stop/start/switch a medicine.
- No unsupported claim that a generic is safer, better, identical in clinical outcome, or currently in stock.
- No invented price, discount, manufacturer, regulatory approval, or availability.
- Never omit the fixed closing sentence: “Confirm with your doctor or pharmacist.” The application should append this sentence in code.

13.4 Safer fallback

Use templates by default. If the explainer model times out or fails validation, show a deterministic template assembled from verified database fields. Post-filter generated text for unsafe advice and unsupported claims.

14. Privacy, auditability and model documentation

- Do not store raw medicine photos by default. If temporary processing storage is unavoidable, delete it promptly and document the retention window.
- Do not collect patient identity, diagnosis, or prescription history for the ML task.
- Use anonymous scan IDs for diagnostics; avoid placing personal data in logs.
- Store source provenance and retrieval dates for catalogue records and prices.
- Version extraction prompts, model/provider versions, schema versions, normalization rules, resolver thresholds, and evaluation datasets.
- Keep a correction feedback queue for human review; never feed unreviewed user corrections directly into training or live matching.

14.1 Model card checklist

- Intended use and explicitly prohibited use.
- Model/provider/version, prompt version, schema version, and date tested.
- Training data statement: pretrained third-party model; no project-specific model training in v1.
- Evaluation set composition, split methodology, metrics, known failure modes, and limitations.
- Confidence/abstention policy and known risks.
- Data retention, source provenance, and update procedure.

15. Build priority and definition of done

P0 — required for the demo

- ■ Image input validation and basic blur/brightness/glare checks.
- ■ Vision-model extraction returning schema-valid JSON and nulls for unreadable fields.
- ■ Catalogue normalization and exact + fuzzy brand resolution.
- ■ Critical-field cross-check and confirmation/abstention gate.
- ■ Deterministic exact/close/none boundary; ML cannot override it.
- ■ Golden set and repeatable evaluation report.
- ■ Error states and no fabricated prices/matches.

P1 — add after P0 is stable

- ■ OCR cross-check for important printed fields.
- ■ Barcode/QR decoding and catalogue lookup.
- ■ Vector retrieval with pgvector, only if it improves measured resolver performance.
- ■ Hindi explainer templates/LLM, latency and cost dashboards.

P2 — stretch only

- ■ Small medicine-text-region detector to crop composition lines.
- ■ On-device OCR/quality model for better offline behavior.

■ Active-learning workflow for reviewed user corrections.

Do not start P2 until P0 passes the release gates. The goal is a safe working system, not the largest number of AI components.

16. Glossary: technical terms in simple English

Term	Plain-English meaning in this project
AI / ML	Software that learns patterns or uses learned models to interpret inputs. Here, it helps read the pack.
VLM (Vision-Language Model)	An AI model that understands images and follows text instructions.
LLM	A large language model that generates or interprets language.
OCR	Optical Character Recognition: converts image text into characters.
Pretrained model	A model trained earlier on broad data; we use it instead of training from scratch.
Inference	Running a trained/pretrained model on a new image.
Fine-tuning	Further training a pretrained model on a narrower dataset; not required for v1.
Prompt	Instructions given to a model.
Structured output / JSON schema	A fixed format that the model's response must follow.
Pydantic validation	Python checks that the returned data has the required fields and types.
Confidence score	A numerical signal about how certain a system thinks a field is; not proof.
Calibration	Checking whether confidence scores correspond to actual correctness.
Abstention / fail closed	Refusing to make a claim when evidence is not strong enough.
Fuzzy matching	Finding strings that are similar despite spelling differences.
Embedding	A list of numbers representing text so similar texts can be retrieved.
Vector search	Searching for nearby embeddings to find semantically similar text.
pg_trgm	PostgreSQL extension that compares text using overlapping character groups.
pgvector	PostgreSQL extension for storing and searching embeddings.
Entity resolution	Deciding which real catalogue record a noisy name refers to.
Canonical ID	A stable database identifier used even when names have aliases.
Normalization	Converting equivalent representations into a consistent format, such as mcg to micrograms.
ETL	Extract, Transform, Load: bring data in, clean it, and load it into the database.
Golden set	A small, carefully labelled test collection used as the reference truth.
Ground truth	The manually verified correct answer used to evaluate the system.
Precision	Of the matches the system claims are correct, how many really are correct?
Recall	Of all the true matches, how many did the system find?
False positive	The system says a match is valid when it is not.
Latency	How long the system takes to produce a result.
p50 / p95 latency	Median response time / response time that 95% of requests meet or beat.
Data leakage	When information from evaluation/test data accidentally helps the system during development, making results look better than they are.
Deterministic rule engine	Ordinary code that gives the same answer for the same verified inputs.
Active learning	Prioritizing uncertain examples for a human to label so the system can be evaluated or improved.
Model card	A document describing a model's purpose, evaluation, limits, and risks.

17. Acceptance checklist

■ A blurry/dark/glare-heavy image gets useful retake guidance.

■ A non-medicine photo never returns a medicine match.

■ Unreadable composition or strength becomes null/unknown, never a guess.

- ■ Every critical field is schema-validated and has confidence/provenance.
- ■ OCR/model/catalogue conflicts block automatic matching.
- ■ Brand resolution returns ranked candidates and evidence, not an unsupported certainty claim.
- ■ Fuzzy or vector similarity alone can never produce EXACT.
- ■ The matcher checks every ingredient, strength, form, release type, and route required by the parent PRD.
- ■ No false EXACT result appears in the golden set; any occurrence blocks release.
- ■ At least 90% per-field extraction accuracy is reported for the required fields on 50+ photos.
- ■ Brand resolver top-1 accuracy is measured against the curated brand list.
- ■ p50/p95 latency and per-scan model cost are measured.
- ■ The app can safely fall back to manual search/template explanation.
- ■ Raw photos are not retained by default and logs avoid personal data.
- ■ Every result preserves the mandatory doctor/pharmacist confirmation.

18. Instructions for an AI coding agent

Implement the ML subsystem described in this document inside the existing project. First inspect the repository, existing API contracts, environment, and data files; do not overwrite existing work without understanding it.

- • Create a short implementation plan and identify which required datasets and API keys are present or missing.
- • Build Pydantic schemas and deterministic normalization/tests before connecting a vision provider.
- • Implement image quality checks, provider abstraction for vision extraction, strict schema validation, OCR/barcode adapters, resolver, and safety gate as separate modules.
- • Keep provider keys in environment variables. Add a .env.example with placeholder values only.
- • Use the existing catalogue as source data after inspecting columns, licence, provenance, and quality. Preserve raw files and create reproducible ETL scripts.
- • Do not fabricate dataset rows, medicine prices, catalogue matches, model results, or evaluation numbers. If data is missing, report the blocker and create tests using clearly marked synthetic fixtures only.
- • Do not train or fine-tune a model unless explicitly requested after baseline evaluation proves it necessary.
- • Build the golden-set evaluation harness before tuning thresholds. Never tune on the final held-out test set.
- • Write unit tests for combination medicines, unit conversion, SR/IR mismatch, missing strength, conflicting OCR/VLM values, malformed JSON, timeout, and no-match behavior.
- • Generate README.md, MODEL_CARD.md, an evaluation report, and a list of known limitations.
- • Before calling the ML layer complete, show test results and a sample scan response, and explicitly state whether each acceptance criterion passed or failed.

Never sacrifice safety to make the demo look successful. If a medicine cannot be verified, return a clear “could not verify” state and allow manual search or pharmacist confirmation.

Appendix A. Source alignment and assumptions

This ML PRD follows the uploaded Sasti Dawai PRD v1.0, particularly section 6 (canonical medicine model and exact-equivalent rules), section 8 (architecture and stack), section 9 (ML/AI pipeline, extraction schema, explainer guardrails, evaluation), and section 10 (catalogue, schema, ETL, provenance). The parent PRD defines the product requirements; this document expands them into implementable ML requirements and beginner-friendly explanations.

The parent PRD proposes a vision-capable model with strict JSON output, OCR/barcode cross-check, exact → fuzzy → vector brand resolution, confidence gating, deterministic matching, optional explainer LLM, and evaluation on 50–100 labelled strip/box photos.

Additional implementation detail in this document—such as the state-machine names, suggested module layout, example API response, and detailed glossary—is a concrete engineering proposal consistent with the parent PRD, not a claim that these components already exist.

Data sizes of approximately 2,439 Jan Aushadhi rows and 253,973 Indian Medicine Dataset records come from prior project discussion. Verify the actual files, schema, licence, and row counts before implementation.